URBANI

Radar UrbiNexus -- DPI 2: Delivery Status & Client Requirements

What is delivered, what is pending, and what we need from Urbani

In one paragraph

All 14 planned DPI 2 deliverables are built and tested in our own environment. Connecting them to Urbani's real systems is pending the inputs listed in section 5. This document shows each deliverable, its honest status, and exactly what we need from your team.

Table of Contents

1. **Executive Summary** 3
   - The three most important asks 3
2. **How to Read This Document** 3
   - Status legend 3
   - Data-honesty principle (kept from DPI 1) 3
3. **Delivery Status by Deliverable** 4
   - 3.1 Security & access 5
   - 3.2 Log management & export 5
   - 3.3 Alerts & notifications 5
   - 3.4 AI Copilot 5
   - 3.5 Monitoring & health 6
   - 3.6 Experience 6
   - 3.7 Platform & operations 6
4. **Mapping to the DPI 2 Success Criteria** 6
5. **What We Need From Urbani** 7
6. **Decisions We Will Ask You to Confirm** 9
7. **Test Evidence in Plain Language** 10
8. **Rollout Plan** 11
9. **Known Limitations and Recommendations** 11

Appendix A: Target Architecture Diagram 12

Appendix B: DRAFT Role Permission Matrix 12

Appendix C: Glossary 13

1. Executive Summary

DPI 2 turns the DPI 1 dashboard into a secure, multi-user control center. It adds roles and sign-in protection, an audit trail, advanced log search and export, automatic alerts by e-mail, Slack and Teams, chat feedback and history, a health panel, Urbani branding with dark mode, and a deployment path that runs on more than one instance. It stays a read-only assistant with a human in the loop.

> **Headline status**
> All 14 planned deliverables are built and tested in our environment. Connecting them to Urbani's real systems is pending the inputs listed in section 5. Nothing has been run against Urbani's AWS account yet.

14 / 14	14 / 14	14 of 14	32
Deliverables built	Verified in our own test environment	Awaiting Urbani input for full live operation (7 need access to Urbani systems, 7 need decisions or assets)	Inputs requested in section 5 (10 are blockers for live)

The three most important asks

- **1. AWS access and deployment approval:** account, region, permissions, and approval for a separate parallel test site. Nothing else can go live without it.
- **2. Connection details for your systems:** identity-provider admin access, log-group access and health-check URLs for QA, Main and Payment, and the alert e-mails and Slack/Teams webhooks.
- **3. Security clean-up and sign-offs:** rotate the shared Urbani API key, and sign off the permission matrix and the draft policy defaults (section 6).

2. How to Read This Document

Status legend

Status	What it means
Built and tested	The feature is built. We tested it in our own environment, using local stand-ins where a real Urbani or AWS service is needed.
Awaiting Urbani input to go live	It cannot run against your real systems until you give us access, details or credentials.
Decision needed	We chose a draft default so work could continue. Please confirm it or change it.

Data-honesty principle (kept from DPI 1)

Every screen carries a provenance label. LIVE means real data from a real source. MOCK means sample data, clearly marked. WAITING FOR INTEGRATION means the source is not connected yet, so the screen says so instead of guessing. Nothing is ever fabricated, including logs, scores and AI answers.

The assistant is read-only. It explains and recommends. It never runs a fix on its own. A person decides and acts.

What "tested" means here

Where a feature needs a real service (AWS, Cognito, Google, Slack, Teams, SNS), we tested against a local stand-in or a simulated service. We do not claim any of these work against the real service until Urbani provides access and we verify them (see section 7).

3. Delivery Status by Deliverable

#	Deliverable	What you get	Status	Still needed to go live
1	**Persistence (DynamoDB)**	Users, audit, alerts and chat are stored in a managed AWS database. This removes the single-instance limit, so 2 or more instances can run.	**Built and tested** **Awaiting Urbani input**	AWS account and region, permission to create the database, approval to deploy.
2	**Role-based access (RBAC)**	Four roles: Super Admin, Infrastructure Administrator, QA Operator, Reader/Auditor. Plus a user-management page.	**Built and tested** **Decision needed**	Sign-off of the permission matrix (Appendix B).
3	**Audit log**	Who did what, when, with what result, for configuration changes and sensitive queries. Searchable Audit page.	**Built and tested** **Decision needed**	List of "sensitive" actions and the retention period.
4	**Log search & filters**	Filter by severity, server, date range, text and regex, with paging. A CloudWatch Logs Insights connection is built and switched off.	**Built and tested** **Awaiting Urbani input**	Log-group names and read access for QA, Main, Payment (or an extension of the Urbani /logs endpoint).
5	**MFA + SSO**	Authenticator-app MFA with recovery codes. Single sign-on (OIDC, e.g. Google Workspace via Cognito) built and switched off.	**Built and tested** **Awaiting Urbani input**	Identity-provider admin access, group-to-role mapping, SSO vs authenticator decision, MFA role policy.
6	**Alerts to email / Slack / Teams**	Rules by severity and service, repeat-notification control, test-send, delivery log. Dry-run mode shows what would be sent.	**Built and tested** **Awaiting Urbani input**	Alert e-mail addresses, Slack/Teams webhook URLs, real rules, confirmation of the design (section 5).
7	**Chat feedback + history**	Like/dislike on every answer, previous conversations can be reopened. Each user sees only their own history.	**Built and tested** **Decision needed**	Retention period and privacy stance for stored questions.
8	**AI latency metric**	Each answer shows how long it took, end to end (including the Urbani chat service). Stored with the answer.	**Built and tested** **Decision needed**	Acceptable response time for live chat (real answers are slower than our test stand-in).
9	**Health / uptime panel**	Status, uptime and history per target. QA, Main and Payment show WAITING FOR INTEGRATION until configured.	**Built and tested** **Awaiting Urbani input**	Health-check URLs and network reach for QA, Main, Payment.
10	**Branding + dark mode**	Light, dark or system theme, remembered per user. Logo and palette are in place as placeholders.	**Built and tested** **Decision needed**	Official logo (SVG/PNG), palette (hex), dark-mode guidance.

#	Deliverable	What you get	Status	Still needed to go live
11	**Export CSV / JSON / PDF**	Download logs (CSV, JSON, PDF), alerts (CSV, JSON) and a PDF incident report. Every export is audited.	**Built and tested** **Decision needed**	Export size limits, whether Reader/ Auditor may export, cloud file storage for large exports.
12	**Custom dashboard layout**	Each user can reorder, hide and reset dashboard widgets. The layout is saved per user.	**Built and tested** **Decision needed**	Which widgets may be personalised.
13	**Load & performance tests**	Repeatable load scripts, a 200,000-line synthetic log test and a browser smoothness test.	**Built and tested** **Awaiting Urbani input**	Agreed performance targets, expected users, permission and a dedicated environment to test in.
14	**Deployment readiness**	Scripts and settings to deploy DPI 2 as a separate parallel site, with 2 or more instances. Safe by default (dry-run).	**Built and tested** **Awaiting Urbani input**	Approval to deploy, operator access, hostname and DNS approval, rotated Urbani API key.

3.1 Security & access

- Four roles control what each person can see and do. A Reader/Auditor cannot change configuration. This is enforced on the server and covered by automated tests per role.
- People can protect their sign-in with an authenticator app and recovery codes. Single sign-on is built and waits for your identity provider.
- Every configuration change and sensitive query is written to an audit log: who, what, when, result.
- The permission matrix is a draft until you sign it off (Appendix B).

3.2 Log management & export

- Filter logs by severity, server, date range, text and regular expression, with paging.
- Download results as CSV, JSON or PDF, plus a PDF incident report. Each file states its source and whether it was cut short.
- Exports are limited in size, rate-limited and audited. Spreadsheet formulas in log text are neutralised.
- Full-stream search over Main and Payment waits for log access (section 5).

3.3 Alerts & notifications

- Rules decide which severities and services notify, per channel: e-mail, Slack, Teams.
- A repeat-notification window stops the same incident from flooding channels.
- An administrator can send a test message per channel, and see a delivery log. Dry-run mode shows what would be sent without sending anything.
- Delivery to real e-mail, Slack and Teams is not yet verified. It needs your addresses and webhooks.

3.4 AI Copilot

- Every answer has a like/dislike. A quality summary shows satisfaction and response times without exposing individual conversations.
- Previous conversations can be reopened. Each user sees only their own. Even a Super Admin cannot read someone else's chat.
- Each answer shows how long it took, end to end.

3.5 Monitoring & health

- A Service Health page and a dashboard strip show status, uptime and history per target.
- The dashboard's own service is checked today. QA, Main and Payment show WAITING FOR INTEGRATION until you give us health-check URLs. They are never shown as live without a real check.
- Health probes are guarded so they cannot be pointed at internal cloud addresses by mistake.

3.6 Experience

- Light, dark or system theme, remembered for each user. Colour contrast is checked automatically.
- Logo and palette are placeholders until Urbani provides the official assets.
- Each user can reorder, hide and reset dashboard widgets. The layout is saved per user and works on narrow screens.

3.7 Platform & operations

- The database layer can run on DynamoDB. The existing database stays the default, so DPI 1 behaviour is unchanged.
- Deployment scripts create DPI 2 as a separate site with 2 or more instances across 2 availability zones. They do nothing unless explicitly told to apply, and they refuse to touch DPI 1.
- Load and performance scripts are ready, with placeholder targets. The deployment scripts have not yet been run against a real AWS account.

4. Mapping to the DPI 2 Success Criteria

The eight success criteria come from the DPI 2 scope document. Each row says what we can show today and what remains.

Success criterion	Status	Evidence we have today	What remains
1. Sign in with SSO/MFA and operate only with your role's permissions. A Reader/ Auditor cannot change configuration.	**Built and tested** **Awaiting Urbani input**	MFA with authenticator app is complete. SSO via Google Workspace is built and tested against a simulated identity provider. Role rules are enforced on the server and tested per role.	Urbani's identity-provider access, group-to-role mapping, and sign-off of the permission matrix.
2. Configuration actions and sensitive queries appear in the audit log with user, action and time.	**Built and tested** **Decision needed**	Each event records user, role, action, target, time and result. Audit page with filters. Retention job. Tested on both database types (the DynamoDB type on a local test instance).	List of sensitive actions and retention period. Not yet run on Urbani's real DynamoDB.
3. Logs can be filtered by severity, server, date, searched by text/regex, and exported to CSV, JSON and PDF.	**Built and tested** **Awaiting Urbani input**	All filters, regex search and the three export formats work on the current sources. The Logs Insights connection is tested with a simulated service only.	Log-group access for QA, Main, Payment for full-stream search. Until then the default source returns a limited window, so exports may be marked as cut short.
4. A critical failure generates an alert by e-mail and in Slack or Teams.	**Built and tested** **Awaiting Urbani input**	Dry-run and test-send proven with simulated channels. Rules, repeat-notification control and delivery log tested.	Real delivery needs e-mail addresses and Slack/Teams webhooks. Not yet verified against real channels.
5. Each chat answer has like/ dislike, past sessions can be consulted, and latency is shown.	**Built and tested** **Decision needed**	Feedback, session history and a latency figure per answer all work against our test stand-in for the chat service.	Retention and privacy decision. Real latency not yet measured against the live Urbani chat service.

Success criterion	Status	Evidence we have today	What remains
6. The status panel shows QA, Main and Payment health. Without source access it is MOCK or WAITING FOR INTEGRATION, never LIVE.	**Awaiting Urbani input**	The panel and probes are built and tested against local servers. QA, Main and Payment show WAITING FOR INTEGRATION until health endpoints are provided. They are never shown as live.	Health-check URLs and network reach for QA, Main and Payment.
7. The interface stays smooth in a load test with massive log generation.	**Built and tested** **Decision needed**	Tests built and run on a development machine, including 200,000 synthetic log lines. The screen stayed smooth with 50,000 lines.	Thresholds are placeholders until Urbani sets them. Tests have not run on AWS.
8. The DPI 1 provenance model is preserved. Nothing is fabricated.	**Met** **Built and tested**	LIVE / MOCK / WAITING FOR INTEGRATION labels on every surface. Incident confidence is derived from severity and labelled "derived". The assistant stays read-only.	Main and Payment stay labelled until they are connected.

5. What We Need From Urbani

This is the most important section. 32 inputs, grouped by topic. Priority: "Blocker for live" means the feature cannot run for real without it. "Needed before go-live" means it must be settled before users start. "Nice to have" can follow later. Owners are suggested roles, not named people.

#	What we need	Why / what it unlocks	Items	Priority	Suggested owner
Access & accounts					
1	AWS account, region and deployment permissions for DPI 2, including approval to deploy and operator access.	Lets us create the database, secrets store, service and alerts topic in Urbani's account.	1, 6, 9, 14	**Blocker for live**	AWS account owner
2	Approval for a parallel test site on its own hostname (we propose dpi2.urbani.io), DNS approval and confirmation of the network and load-balancer details.	DPI 2 runs next to DPI 1. Nothing existing is disturbed.	14	**Blocker for live**	AWS account owner
3	Rotate the previously shared Urbani API key and send the new one through a secure channel.	Closes a security carry-over from DPI 1. We store it in the secrets store only.	4, 8, 14	**Blocker for live**	Security lead
4	A strong administrator password. We generate it and store it in the secrets store. Please confirm the DPI 1 default admin password is rotated or DPI 1 is retired.	Removes the default credential before real users arrive.	2, 14	**Blocker for live**	Security lead
5	Log-group names and read access for QA, Main and Payment, or an extension of the Urbani /logs endpoint.	Enables full-stream search. Today only lines the Urbani service flags are searchable.	4, 11	**Needed before go-live**	AWS account owner
Identity & security					
6	Admin access to Google Workspace (or the chosen identity provider) and the group-to-role mapping.	Completes single sign-on. People sign in with their corporate account and get the right role.	2, 5	**Blocker for live**	Google Workspace admin

#	What we need	Why / what it unlocks	Items	Priority	Suggested owner
7	Decision: single sign-on, authenticator app, or both.	Decides whether the identity-provider work is needed at all.	5	**Needed before go-live**	Security lead
8	Which roles must use MFA. Draft: optional for everyone.	Turns on enforced MFA for the roles you name.	5	**Needed before go-live**	Security lead
9	Sign-off of the permission matrix (Appendix B).	Confirms who can see and do what. Role rules are enforced on the server.	2	**Needed before go-live**	Product owner
10	Whether Reader/Auditor may export, and whether auditors may ever read chat content (draft: no).	Sets export and chat privacy rules for the audit role.	2, 7, 11	**Needed before go-live**	Security lead
Notification channels					
11	Alert e-mail addresses, and who confirms the subscription e-mails.	Required for real e-mail alerts.	6	**Blocker for live**	Operations lead
12	Slack and/or Teams webhook URLs, sent through a secure channel (not e-mail or chat).	Required for real Slack/Teams alerts. Stored in the secrets store.	6	**Blocker for live**	Operations lead
13	Alert rules: which severities and services notify, the repeat-notification (dedup) window and the hourly cap. Draft: High and above, 30 minutes, 30 per hour.	Avoids alert fatigue. Defaults can start immediately.	6	**Needed before go-live**	Operations lead
14	The public address of the dashboard, for links inside alerts.	Alert messages link straight to the incident.	6	**Needed before go-live**	Operations lead
Monitoring targets					
15	Confirm what "QA, Main, Payment" mean as monitored targets (servers, services or environments, and where they run).	We cannot probe or search them without a clear definition.	4, 9	**Blocker for live**	Operations lead
16	Health-check URLs for QA, Main and Payment, with the expected response and any sign-in needed.	Switches the panel from WAITING FOR INTEGRATION to live status.	9	**Blocker for live**	Operations lead
17	Network reachability from the DPI 2 service to those URLs (or an internal route).	Probes must be able to reach the targets.	9	**Blocker for live**	AWS account owner
Branding					
18	Official logo (SVG and PNG, light and dark versions).	Replaces the placeholder logo.	10	**Needed before go-live**	Brand/marketing
19	Brand palette (hex values) and dark-mode guidance.	Replaces the placeholder colours and checks contrast.	10	**Nice to have**	Brand/marketing
Policies & decisions					
20	Audit-log retention (draft 365 days) and which actions count as "sensitive".	Sets how long audit records are kept and what is flagged.	3	**Needed before go-live**	Security lead
21	Chat-history retention (draft 90 days) and your stance on storing users' questions.	Questions are stored as plain text per user today.	7	**Needed before go-live**	Security lead
22	Notification-log retention (draft 90 days).	Sets how long delivery records are kept.	6	**Nice to have**	Operations lead

#	What we need	Why / what it unlocks	Items	Priority	Suggested owner
23	Health-history retention (draft 30 days).	Sets how much uptime history is kept.	9	**Nice to have**	Operations lead
24	Export limits (draft 10,000 rows for CSV/JSON, 2,000 for PDF), retention of exported files, and whether large exports should use cloud file storage.	Protects the service from very large downloads.	11	**Nice to have**	Product owner
25	Performance targets: response times, error rate, expected concurrent users, peak log volume, and an acceptable response time for live chat.	Today's thresholds are placeholders. Real targets make the tests meaningful.	8, 13	**Needed before go-live**	Operations lead
26	Permission to run load tests in a dedicated environment.	We never load-test production or Urbani's QA service without approval.	13	**Needed before go-live**	AWS account owner
27	Which dashboard widgets may be personalised, and whether an admin default layout is wanted.	Defines the personalisation scope.	12	**Nice to have**	Product owner
28	Confirm the alert design: the dashboard service sends alerts itself (section 6). The original DynamoDB-Streams + Lambda design remains available.	One decision that shapes how alerts run in production.	6	**Needed before go-live**	Product owner
Environment & rollout					
29	A named business sponsor.	The scope document leaves this open. Needed for acceptance.	All	**Needed before go-live**	Product owner
30	Test users for each of the four roles.	Needed for user acceptance testing.	2, 5	**Needed before go-live**	Product owner
31	A cutover date and window, and the approval to retire DPI 1 afterwards.	DPI 1 keeps running untouched until you approve.	14	**Needed before go-live**	Product owner
32	Decision on DPI 1 data: start fresh or copy it into DPI 2.	DPI 1 data lives on a single instance; DPI 2 uses the new database.	1, 14	**Needed before go-live**	Operations lead

> **Please send secrets through a secure channel**
> Webhook URLs, the rotated API key and any credentials should never be sent by e-mail or chat. We will agree a secure hand-over and store them in the AWS secrets store only.

6. Decisions We Will Ask You to Confirm

We chose these draft defaults so work could continue. Each can be changed, mostly by configuration.

Decision	Draft default we chose	Your options
Permission matrix	Four roles as in Appendix B. Reader/Auditor can view, export and use the chat but cannot change anything.	Adjust any role. For example, remove export or chat from Reader/Auditor.
MFA policy	Optional for everyone.	Require MFA for chosen roles, or for all.
Sign-in method	Local sign-in with optional authenticator app. SSO built and switched off.	SSO only, authenticator only, or both.

Decision	Draft default we chose	Your options
Alert design	The dashboard service sends alerts itself. It works with the current database and DynamoDB, and is safe with several instances.	Keep it, or use the originally proposed DynamoDB-Streams + Lambda design. The alternative is available.
Alert rules	High and above, all channels, same incident notified once per 30 minutes, at most 30 per hour.	Any severity, service or window.
Retention	Audit 365 days. Chat history 90 days. Notification log 90 days. Health history 30 days.	Any period, or keep forever.
Export limits	10,000 rows for CSV/JSON. 2,000 rows for PDF.	Other limits, or cloud file storage for large exports.
Parallel site	DPI 2 runs on its own hostname (proposed dpi2.urbani.io) next to DPI 1.	Another hostname.
Chat privacy	Questions stored per user. Auditors cannot read chat content.	Turn history off, shorten retention, or allow auditor access.
DPI 1 data	Not copied automatically.	Start fresh, or copy DPI 1 data into DPI 2.

7. Test Evidence in Plain Language

Area	What we ran	Result
Automated tests	The full suite of automated checks on the application.	641 tests pass. A further 104 database tests also pass against a local DynamoDB test instance.
Browser checks	A browser walked through the screens in light and dark mode.	43 checks across 15 screens passed. It found one real crash on the dashboard, which we fixed.
Load: realistic	50 simulated users on a development machine, 50,000 log lines, with pauses like real users.	No errors. 95th-percentile response times: reading 10 ms, search 18 ms, regex search 61 ms, export 99 ms, chat on a stand-in 24 ms.
Load: large logs	200,000 log lines, 25 users, search-heavy.	No errors. 95th-percentile: search 17 ms, regex 163 ms, export 46 ms. On a busy machine, up to about 0.5 s for search and 1.4 s for regex.
Load: no pauses	Pushing 50 users with no pause on 200,000 lines.	Regex searches slowed to 1.4 - 2.3 s at the 95th percentile and two runs went past the draft limits. See section 9.
Browser smoothness	Logs page with 50,000 lines, with and without background load.	First rows in about 0.4 - 0.5 s. No freeze longer than 50 ms during paging, filtering and typing.
Security checks	Secrets scan, role-enforcement tests, input hardening.	Secrets scan clean. Role rules tested per role. Regex safety limits, export formula-injection protection and a webhook address allow-list are in place and tested.

> **Important: these are development-machine numbers**
> The load figures come from a laptop-class development machine using sample data and local stand-ins. They are not AWS production numbers. The pass/fail thresholds are placeholders until Urbani sets real targets. The k6 and AWS Distributed Load Testing tools have not been run.

What has NOT been verified yet

Nothing was run against Urbani's AWS account. The deployment scripts have only been validated locally (dry-run and local containers). These are untested against real services: SSO with Google or Cognito; e-mail, Slack and Teams delivery; CloudWatch Logs Insights; Route 53 and Synthetics monitoring; real DynamoDB; and real chat latency.

8. Rollout Plan

No dates are set. Timing depends on when Urbani's inputs arrive ("Pending Urbani input").

Step	What happens	Done when
0. Collect inputs	We gather the items in section 5 and agree a secure hand-over for secrets.	Blockers for live are received.
1. Parallel deploy	DPI 2 is deployed as a separate site next to DPI 1 on its own hostname. Nothing existing is disturbed.	The DPI 2 site is reachable and healthy on 2 instances.
2. Connect integrations	Each integration is switched on as its input arrives. This is configuration, not new development. Each is checked against a checklist.	Each integration passes its check with real data.
3. Acceptance testing	Test users, one per role, try the system against the success criteria.	Urbani signs off acceptance.
4. Cutover	The main address is pointed at DPI 2 and DPI 1 is retired. Rollback is simple: point the address back.	Urbani approves cutover and retirement.

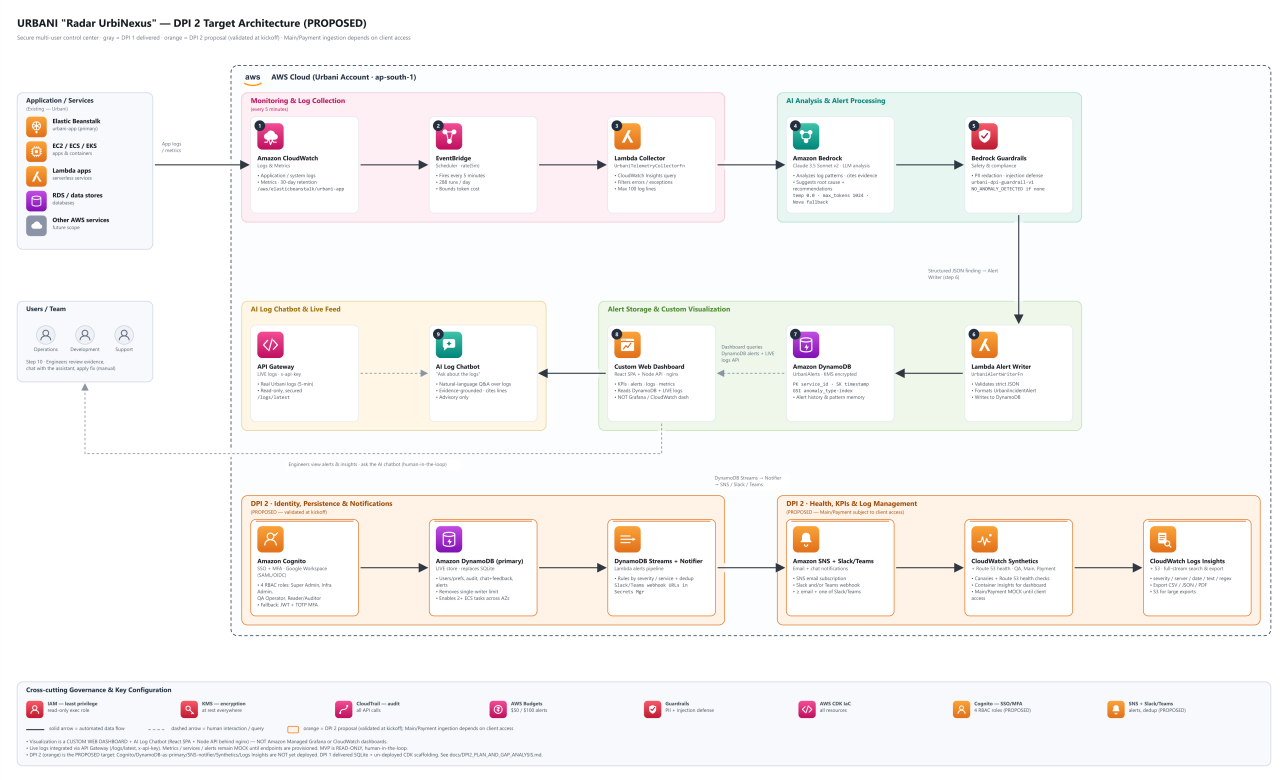
DPI 1 is not touched

DPI 1 keeps running exactly as it does today until Urbani approves cutover. Our deployment scripts refuse to change DPI 1 resources.

9. Known Limitations and Recommendations

- **Run 2 or more instances:** The service handles load on one processor core per instance. DPI 2 is planned for 2 or more instances with automatic scaling.
- **Very large log volumes:** Use CloudWatch Logs Insights for search over large volumes. Our in-service search is fine for moderate volumes, but regex search slows under heavy load.
- **Dashboard summary caching:** If many people watch the dashboard at once, caching the summary for a few seconds would help. We have not turned it on.
- **Regex search is bounded:** On purpose. Patterns are checked for safety and time-limited, so one bad pattern cannot slow the service for everyone.
- **Main and Payment stay labelled:** Anything that depends on Main or Payment data stays marked WAITING FOR INTEGRATION until those systems are connected.
- **Incident confidence is derived:** The "confidence" shown on incidents is calculated from severity and clearly labelled as derived. It is not a model score.
- **Limits per instance:** Request rate limits are kept in memory on each instance, so they multiply with the number of instances.
- **Chat latency:** Real AI answers are slower than our stand-in. Please set an acceptable response time (section 5).

Appendix A: Target Architecture Diagram



Grey shows the DPI 1 base. Orange shows what is new in DPI 2 (identity, database, notifications and health monitoring). Orange services are proposals to confirm with Urbani. Ingestion from Main and Payment depends on access to those sources.

Appendix B: DRAFT Role Permission Matrix

Draft for Urbani sign-off. The same matrix is used by the server and the screens.

Capability	Super Admin	Infra Admin	QA Operator	Reader / Auditor
View dashboard, alerts, logs, metrics, services, AI insights, usage, settings	Yes	Yes	Yes	Yes
Acknowledge alerts	Yes	Yes	Yes	No
Run AI analysis on an incident	Yes	Yes	Yes	No
Use the chat assistant (read-only advice)	Yes	Yes	Yes	Yes (judgement call)
Export logs, alerts and incident reports	Yes	Yes	Yes	Yes
Manage alert rules and send test notifications	Yes	Yes	No	No
Change settings and run a health check now	Yes	Yes	No	No
View the audit log and chat quality summary	Yes	Yes	No	Yes
Manage users and roles	Yes	No	No	No

Chat conversations are private to each user. Nobody, including a Super Admin, can read another person's conversation. Urbani may remove chat or export from Reader/Auditor.

Appendix C: Glossary

Term	Meaning
RBAC	Role-based access control: what a person can do depends on their role.
SSO	Single sign-on: people sign in once with their corporate account.
MFA / TOTP	Multi-factor sign-in. TOTP is the six-digit code from an authenticator app.
Audit log	A record of who did what, and when. It has no edit or delete option, only the retention clean-up.
Dedup	Repeat-notification control: the same incident is announced once per time window.
DynamoDB	Amazon's managed database service. It replaces the single-instance database used in DPI 1.
ECS Fargate	The AWS service that runs the dashboard in containers without managing servers.
Provenance label	The tag on every screen saying where data comes from: LIVE, MOCK or WAITING FOR INTEGRATION.
Logs Insights	The CloudWatch feature for searching the full stream of log lines at scale.